

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнила:

Дьячкова Елизавета

Группа J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задачей данной лабораторной работы является миграция ранее разработанного веб-приложения TicketHub (выполненного в рамках ЛР1 и ЛР2) на фреймворк Vue.JS с созданием полноценного одностраничного приложения (SPA). Основными требованиями являются:

1. Подключение Vue Router для реализации клиентской навигации между страницами без перезагрузки.
2. Реализация работы с внешним API посредством библиотеки axios.
3. Разумное деление интерфейса на компоненты с демонстрацией понимания компонентного подхода.
4. Использование composable-функций для выделения повторяющегося функционала в отдельные файлы.

Ход работы

1. Настройка проекта

Проект создан с помощью Vite и Vue 3. В файле vite.config.js настроен псевдоним @ для удобного импорта из папки src, а также прокси-сервер, перенаправляющий запросы с пути /api на локальный бэкенд из ЛР2 (порт 3001):

```
server: {  
  
  proxy: {  
  
    '/api': {  
  
      target: 'http://127.0.0.1:3001',  
  
      changeOrigin: true,  
  
      rewrite: (path) => path.replace(/^\/api/, ''),  
  
    },  
  
  },  
  
}
```

Базовый URL API вынесен в файлы окружения: в .env.development задан /api (через прокси), в .env.production - прямой адрес http://127.0.0.1:3001.

```
found 0 vulnerabilities
```

```
> tickethub-lab3@0.0.0 dev
> vite
```

```
VITE v6.4.2 ready in 3552 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
13:06:31 [vite] (client) page reload src/composables/useTheme.js
13:16:02 [vite] (client) hmr update /src/App.vue
13:16:25 [vite] (client) hmr update /src/components/AppNavbar.vue
13:16:25 [vite] (client) hmr update /src/views/EventDetailView.vue
13:16:25 [vite] (client) hmr update /src/views/HomeView.vue
13:21:43 [vite] (client) hmr update /src/views/HomeView.vue, /src/views/EventDet
ailView.vue
```

2. Подключение роутера

Для навигации подключён Vue Router. В файле router/index.js описаны все маршруты приложения с использованием вложенной структуры через общий MainLayout:

/ - главная страница с поиском мероприятий

/event/:id - страница мероприятия

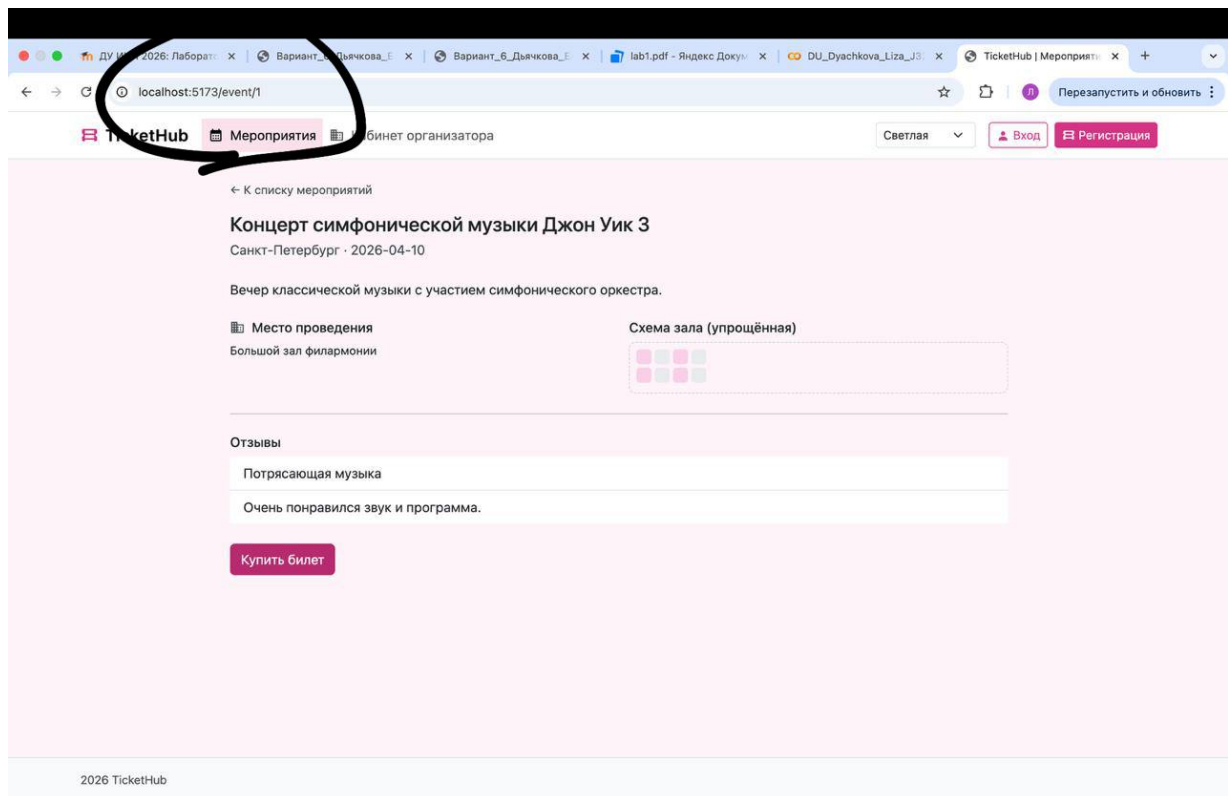
/login - вход

/register - регистрация

/profile - личный кабинет пользователя

/organizer - кабинет организатора

Для защиты страниц реализован navigation guard через router.beforeEach. Страницы с метой requiresAuth недоступны без авторизации и перенаправляют на /login, страницы с guestOnly недоступны для уже вошедших пользователей. Также при каждом переходе автоматически обновляется заголовок вкладки браузера.



3. Работа с внешним API через axios

В файле `api/http.js` создан экземпляр `axios` с базовым URL из переменной окружения. Настроен перехватчик запросов (`interceptor`), который автоматически добавляет JWT-токен из `localStorage` в заголовок `Authorization: Bearer` для каждого запроса, если токен присутствует:

```
http.interceptors.request.use((config) => {
  if (!config.skipAuth) {
    const token = localStorage.getItem('tickethub_auth_token')
    if (token) config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

Флаг `skipAuth: true` используется для запросов, не требующих авторизации (загрузка мероприятий, вход, регистрация). Также реализована вспомогательная функция `unwgar`, которая проверяет статус ответа и

выбрасывает ошибку с понятным сообщением при неудачных запросах.

The screenshot shows the TicketHub application interface on the left and the Chrome DevTools Network tab on the right. The application has a search bar and filters for event type, date, and location. It displays three event cards: a symphony concert, a theatrical production, and a music festival. The DevTools Network tab shows a list of network requests, including index.js, bootstrap.min.css, and various Vue.js components like AppNavbar.vue, EventCard.vue, SearchFilters.vue, useEventsApi.js, events.js, useCity.js, and events.

Name	Status	Type	Initiator	Size	Time
index.js?t=1778494903411	304	script	main.js:5	0.2 kB	5 ms
bootstrap.min.css	304	script	main.js:2	0.2 kB	5 ms
304 Not Modified	304	script	main.js:8	0.2 kB	4 ms
bootstrap_dist_js_bootstrap...	200	script	main.js:9	(disk c...	3 ms
env.mjs	304	script	client:1	0.2 kB	2 ms
chunk-PRB2KXUH.js?v=0abd...	200	script	vue.js?v=7bf2ebc...	(disk c...	3 ms
chunk-HKJ2B2AA.js?v=0abd...	200	script	vue.js?v=7bf2ebc...	(disk c...	1 ms
ToastHost.vue	304	script	App.vue:7	0.2 kB	3 ms
__x00__plugin-vue:export-he...	304	script	App.vue:21	0.2 kB	4 ms
vue-router.js?v=fed64a1a	200	script	index.js:1	(disk c...	2 ms
MainLayout.vue?t=17784945...	304	script	index.js:3	0.2 kB	3 ms
useAuth.js	304	script	index.js:4	0.2 kB	2 ms
?token=dq2IHxgJz7uF	101	webso...	client:802	0.0 kB	Pending
AppNavbar.vue?t=17784945...	304	script	MainLayout.vue:2	0.2 kB	1 ms
http.js	304	script	useAuth.js:2	0.2 kB	1 ms
useToast.js	304	script	ToastHost.vue:24	0.2 kB	1 ms
ThemeSelect.vue?t=1778493...	304	script	AppNavbar.vue:9	0.2 kB	2 ms
bootstrap.js?v=fd6c27fa	200	script	useToast.js:2	(disk c...	1 ms
axios.js?v=24f58885	200	script	http.js:1	(disk c...	2 ms
HomeView.vue?t=177849490...	304	script	index.js:16	0.2 kB	1 ms
EventCard.vue	304	script	HomeView.vue:50	0.2 kB	2 ms
SearchFilters.vue	304	script	HomeView.vue:51	0.2 kB	3 ms
useEventsApi.js?t=17784949...	304	script	HomeView.vue:52	0.2 kB	3 ms
events.js	304	script	HomeView.vue:53	0.2 kB	3 ms
useCity.js	304	script	EventCard.vue:25	0.2 kB	2 ms
events	200	xhr	useEventsApi.js:6	1.2 kB	8 ms
data:image/svg+xml,...	200	svg+xml	Other	(memo...	0 ms
data:image/svg+xml,...	200	svg+xml	Other	(memo...	0 ms

4. Деление на компоненты

Приложение разделено на переиспользуемые компоненты, каждый из которых отвечает за одну задачу:

AppNavbar.vue - шапка с навигацией, динамически отображает кнопки «Вход/Регистрация» или «Личный кабинет» в зависимости от состояния авторизации.

EventCard.vue - карточка одного мероприятия со ссылкой на детальную страницу.

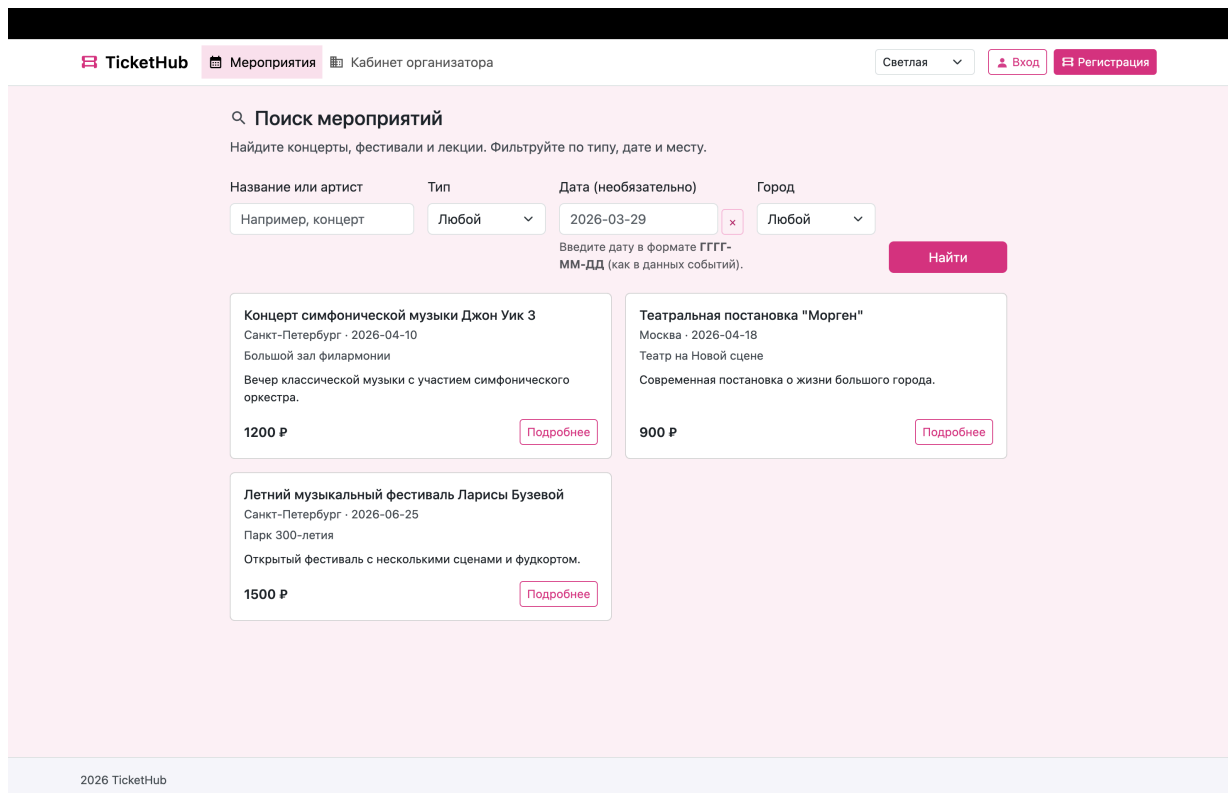
SearchFilters.vue - форма фильтрации мероприятий по названию, типу, дате и городу. При сабмите генерирует событие `search` для родительского компонента.

ThemeSelect.vue - выпадающий список для выбора темы оформления.

ToastHost.vue - контейнер для всплывающих уведомлений Bootstrap Toast.

SeatMapPreview.vue - упрощённая схема зала на странице мероприятия.

Общий каркас страниц (шапка, основной контент, футер) вынесен в `layouts/MainLayout.vue`, который используется через вложенные маршруты роутера.



5. Использование composable

Повторяющийся функционал выделен в пять composable-функций в папке `src/composables`:

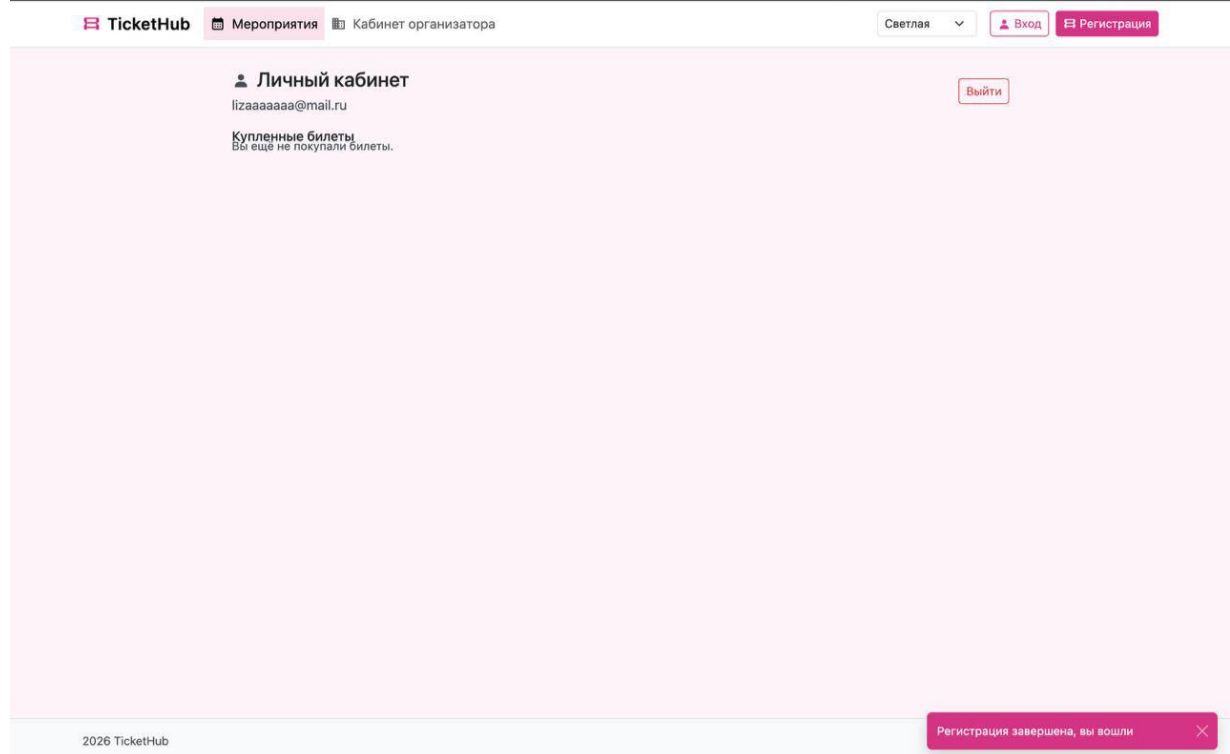
`useAuth.js` - управление авторизацией. Содержит логику входа, регистрации и выхода из аккаунта. Текущий пользователь хранится в реактивном `ref`, благодаря чему все компоненты, использующие `useAuth`, автоматически реагируют на изменение состояния авторизации. Composable используется в `AppNavbar`, `EventDetailView`, `ProfileView` и `OrganizerView`.

`useEventsApi.js` - запросы к мероприятиям. Содержит функции `fetchEventsList` и `getEventByld`, используемые на главной странице, странице мероприятия и в кабинете организатора.

`useTheme.js` - переключение темы оформления. Сохраняет выбор пользователя в `localStorage` и применяет атрибут `data-theme` к корневому элементу страницы. Инициализируется один раз при запуске в `main.js`.

`useToast.js` - управление всплывающими уведомлениями. Реактивное сообщение передаётся в `ToastHost.vue`, а метод `show(text)` вызывается из любого компонента.

useCity.js - вспомогательная функция cityLabel для преобразования кода города в читаемое название. Используется в EventCard, EventDetailView и OrganizerView.



6. Страницы приложения

HomeView.vue - загружает список мероприятий через useEventsApi при монтировании. Реактивный computed displayed фильтрует события на основе состояния формы SearchFilters. Пока идёт загрузка, отображается текст «Загрузка...», при ошибке - подсказка запустить бэкенд.

EventDetailView.vue - загружает мероприятие по ID из параметра маршрута. При нажатии «Купить билет» проверяет авторизацию и отправляет POST-запрос на /tickets. Перезагружает данные при изменении параметра маршрута через watch.

LoginView.vue и RegisterView.vue - формы авторизации, использующие useAuth. После успешного входа/регистрации перенаправляют в личный кабинет.

ProfileView.vue - загружает список купленных билетов через GET /tickets?userId=.... При выходе из аккаунта очищает токен и перенаправляет на главную.

ТicketHub

Мероприятия

Кабинет организатора

Светлая

Личный кабинет

← К списку мероприятий

Концерт симфонической музыки Джон Уик 3

Санкт-Петербург · 2026-04-10

Вечер классической музыки с участием симфонического оркестра.

Место проведения

Большой зал филармонии

Схема зала (упрощённая)

Отзывы

Потрясающая музыка

Очень понравился звук и программа.

Купить билет

ТicketHub

Мероприятия

Кабинет организатора

Светлая

Личный кабинет

Кабинет организатора

Создать событие

Название

Город

Выберите город

Дата

ГГГГ-ММ-ДД

Формат: ГГГГ-ММ-ДД (например, 2026-03-29).

Тип

Концерт

Цена от

0

Добавить событие

Мои события

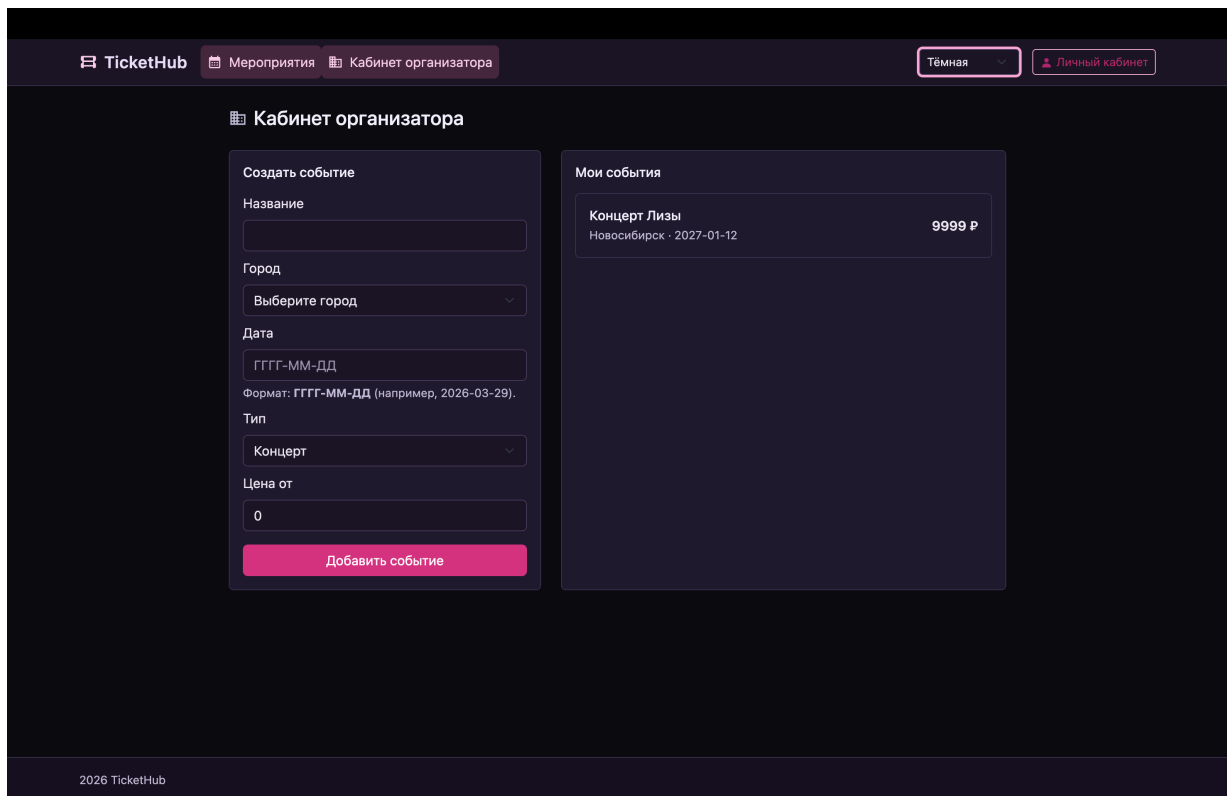
Концерт Лизы

Новосибирск · 2027-01-12

9999 Р

2026 TicketHub

Событие создано



OrganizerView.vue - отображает только события текущего пользователя (фильтр по organizerUserId). Форма создания события отправляет POST-запрос на /events, после чего список обновляется без перезагрузки.

Вывод

В ходе работы приложение TicketHub было полностью перенесено с ванильного JavaScript на фреймворк Vue.JS. Вместо нескольких отдельных HTML-файлов реализовано единое одностраничное приложение с клиентской навигацией через Vue Router. Логика работы с API переведена с fetch на axios с настроенным перехватчиком для JWT-авторизации. Интерфейс разбит на переиспользуемые компоненты, а повторяющийся функционал (авторизация, запросы, тема, уведомления) вынесен в composable-функции. В результате код стал структурированным, поддерживаемым и соответствует современным подходам к разработке фронтенда.

Работа была очень интересной и полезной для меня!